



LeTourneau University Space Program

Advanced Remote Ignition and Evaluation System (ARIES)
Cooper McAllister, Avionics Team Lead

November 14, 2025



Table of Contents

COVER PAGE.....	1
TABLE OF CONTENTS	2
TABLE OF ABBREVIATIONS.....	3
INTRODUCTION	4
PROJECT REQUIREMENTS AND DESIGN DECISIONS	4
PCB DESIGN.....	5
PYRO IGNITION CIRCUITS.....	5
MICROCONTROLLER	7
NRF24L01 RADIO	8
SD CARD	8
BUZZER.....	9
HX711 LOAD CELL AMPLIFIER	9
PCB LAYOUT.....	9
HARDWARE DESIGN AND ASSEMBLY	12
SOFTWARE DESIGN	13
SOFTWARE OVERVIEW	14
FUTURE WORK AND LESSONS LEARNED	16



Table of Abbreviations

Abbreviation	Meaning
Pi	Raspberry Pi Pico
ADC	Analog-Digital Converter
SPI	Serial Peripheral Interface
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
SDK	Software Development Kit
PCB	Printed Circuit Board



Introduction

This document describes the design and production of the Advanced Remote Ignition and Evaluation System (ARIES) for LetSpace's propulsion program, Trepid. This document also includes technical details on the operation of ARIES to assist anyone who may be using it in the future. I (Cooper McAllister) did most of the work on ARIES, with help from Josh Hartman, Grey (Joseph) Marney, and various others. I initiated this project after the previous motor test system, Compact Remote Ignition System (CRIS), was destroyed in a test incident. Since the person who made CRIS did not leave any documentation before graduating, we couldn't simply rebuild CRIS. Therefore, a new system was needed. Additionally, ARIES was designed with integrated data logging, which CRIS lacked.

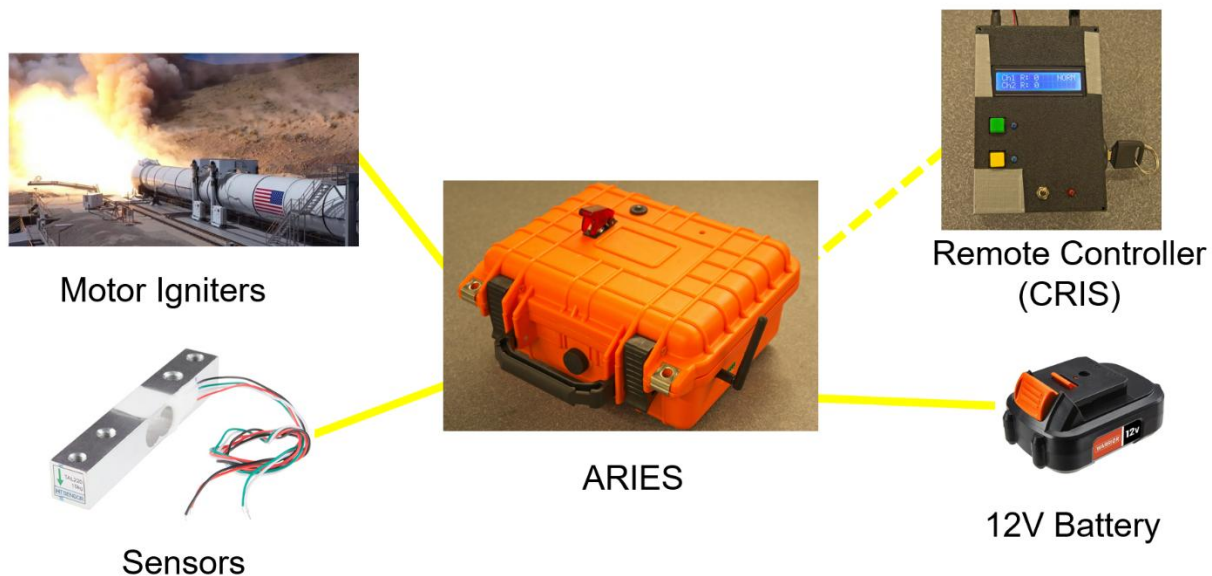
Project Requirements and Design Decisions

I used the following “big picture” design requirements for ARIES:

- Remote ignition of motors for propulsion development
- Data collection from strain gauges and other sensors
- Flexibility and reliability

I decided to reuse the remote control for CRIS, which still mostly worked.

The following diagram shows the general role of ARIES, connecting several different subsystems:



The design was centered around a custom PCB inside a case, with connectors for igniters and sensors. At the beginning of the design process, it was decided that ARIES would have two ignition channels, to allow simultaneous testing of multiple igniters or motors.



PCB Design

The PCB is where most of ARIES's functionality is contained, and it was the first part of ARIES I designed. I used KiCad, of which I had learned the basics in Digital Electronics, but didn't have much experience with prior to this project. I decided to center the design around a [Raspberry Pi Pico](#), since our avionics (STELLA) uses that microcontroller.

Pyro Ignition Circuits

I borrowed the pyro ignition circuits from STELLA, with a few minor changes. A single ignition channel is shown below:

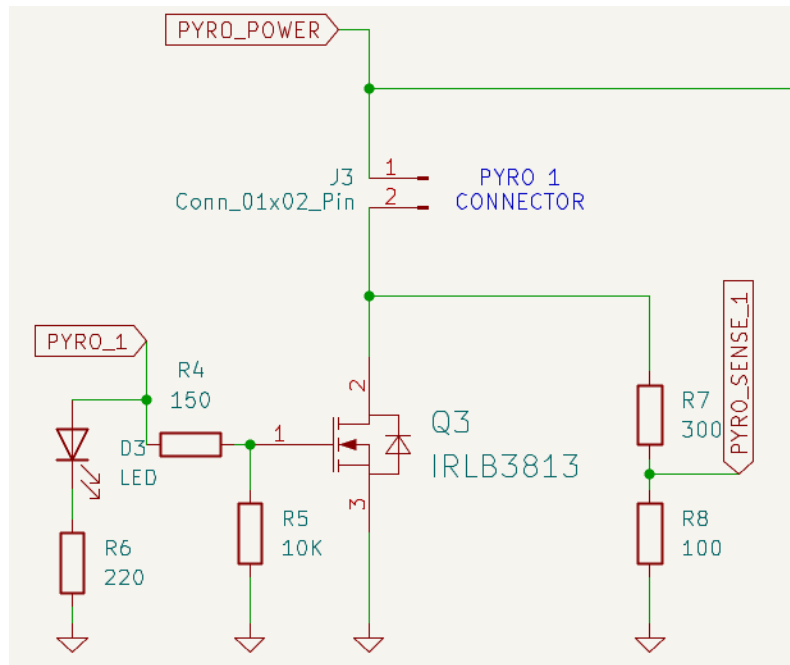


Figure 1: Pyro Ignition Schematic

The pyro power is controlled by an N-channel MOSFET, the [IRLB3813](#). When the PRYO_1 signal, which is controlled by the raspberry pi and tied to the MOSFET gate, goes high, the MOSFET creates a path for current to flow through the igniter. Current will only flow if the arm switch, which connects PYRO_POWER to the battery, is on. The igniter pulls potentially 5-10A, but it burns out (and thus current stops flowing) after $\approx 100\text{ms}$. However, the PCB was still designed to handle large currents flowing through the igniters. PRYO_1 is also tied to an LED, which provides a helpful visual indication of pyro ignition for debugging code. The resistor R4 of 150Ω limits the current flowing into the MOSFET, and R5 of $10\text{k}\Omega$ pulls down the MOSFET gate, keeping the MOSFET off normally.

The pyro channels also include circuitry to measure the resistance of an igniter by passing a small test current through the igniters when ARIES is armed. Below is shown the power input circuitry:

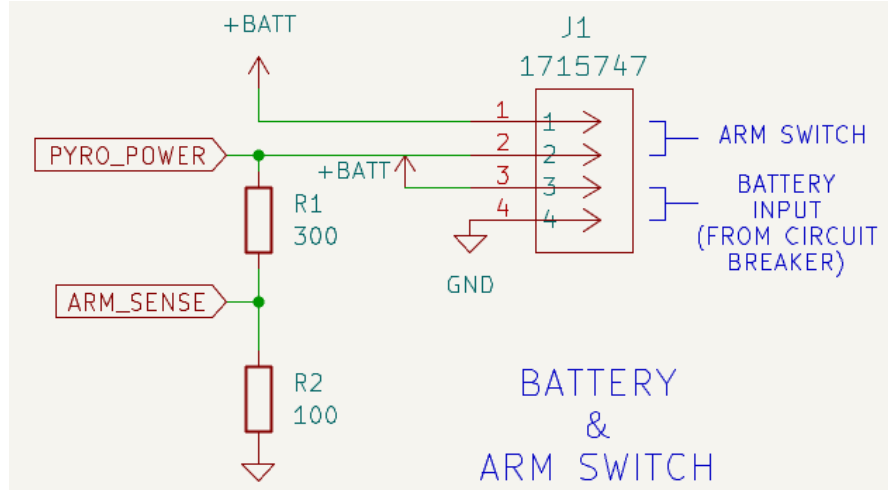


Figure 2: Power Input Circuitry

When the Arm Switch is on, PYRO_POWER will be at the voltage of the battery. ARM_SENSE goes to an ADC channel on the Pi, and the battery voltage can be calculated using voltage division:

$$V_{ARM} = V_{BAT} \cdot \frac{100}{100 + 300} = \frac{1}{4} V_{BAT} \rightarrow V_{BAT} = 4V_{ARM}$$

Similarly, the voltage at the MOSFET drain can be calculated as

$$V_D = 4V_{SENSE}$$

Where V_{SENSE} is the voltage measured at PYRO_SENSE_1.

With these two voltages (V_D and V_{ARM}) it is possible to calculate the igniter resistance as follows, using Ohm's law and KVL, and finding the current through the igniter by finding the current flowing through the 100Ω and 300Ω resistors:

$$R_{IGNITER} = \frac{V_{IGNITER}}{I_{IGNITER}} = \frac{V_{ARM} - V_D}{\frac{V_D}{400\Omega}}$$

As stated before, the same pyro circuitry was used on STELLA, but I changed the resistor values to accomplish two things: (1) ensure only a very small current (<40mA) flows through the igniter while ARIES is armed (so that the igniter does not light until commanded), and (2) keep the voltage at the ADC input below 3.3V, which is the maximum rated voltage for the Pi's ADC.

Microcontroller

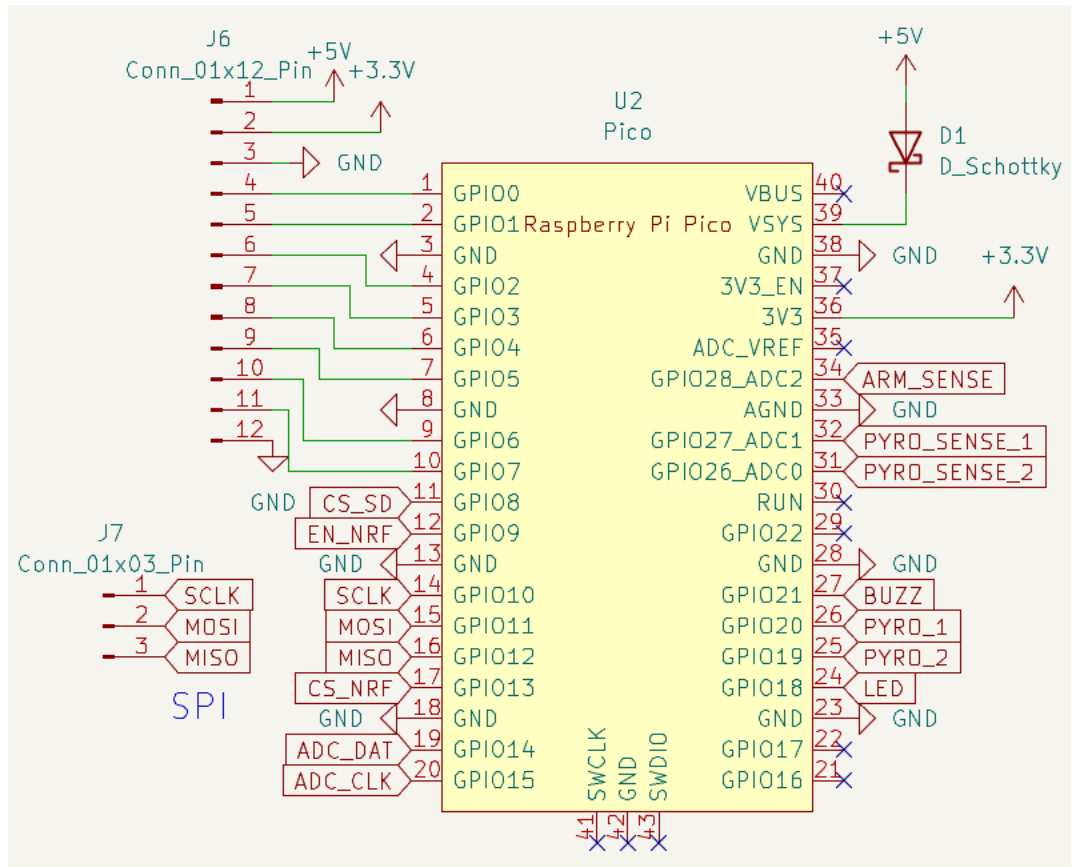


Figure 3: Raspberry Pi Pico Microcontroller Schematic

The Pi is connected to the pyro ignition and sense channels, HX711, SD Card, and NRF24L01 radio. The Pi is powered by a 5V regulator, the [LM7805CT](#). There is a Schottky diode before the Pi so that the battery and USB power can both be connected simultaneously without the USB power feeding back into the rest of the circuitry. There are pin headers for the SPI bus and the rest of the unused pins on the Pi, so that additional peripherals can be added in the future with relative ease.

NRF24L01 Radio

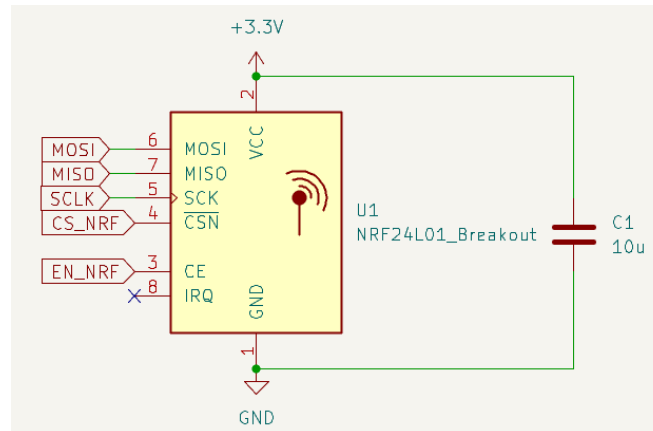


Figure 4: NRF24L01 Radio Schematic

A [breakout board](#) was used for the [NRF24L01](#) radio, a radio commonly used for Arduino hobby projects. A 10µF capacitor was used for decoupling, since this radio is sensitive to power supply fluctuations. The radio is connected to the Pi via SPI.

SD Card

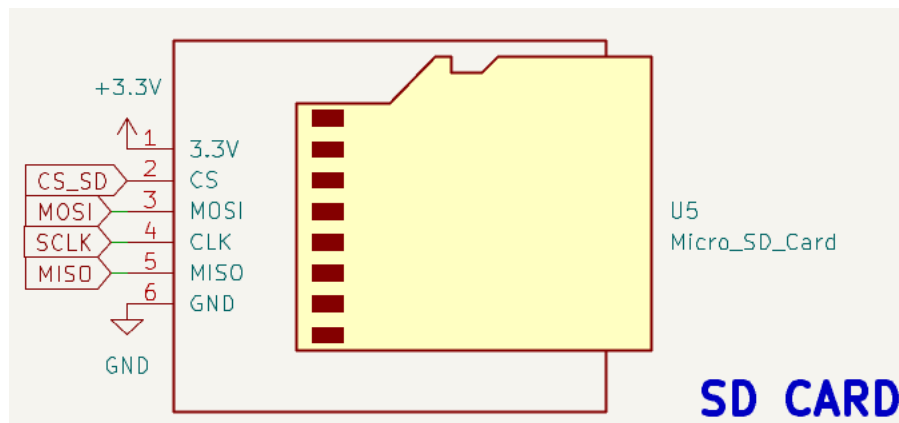


Figure 5: SD Card Schematic

SD Cards use SPI, so connecting the SD Card to the Pi was very straightforward.



Buzzer

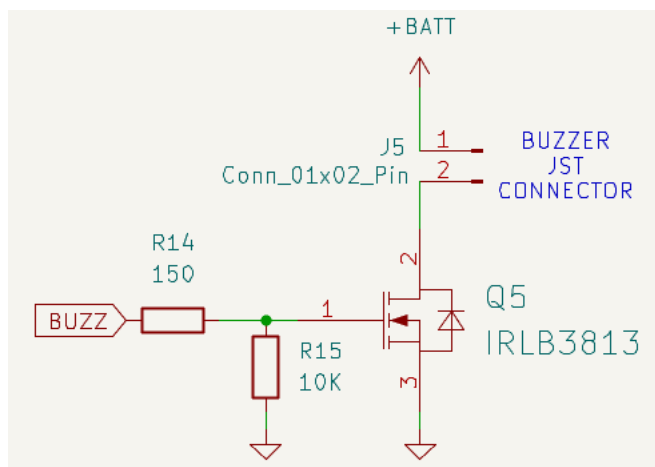
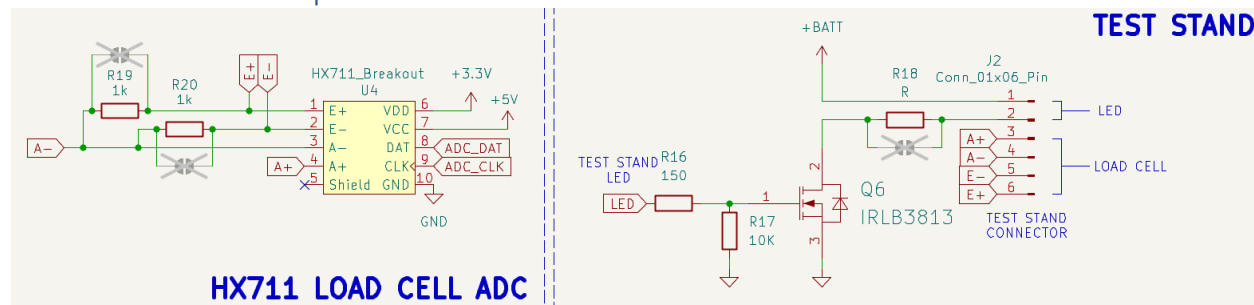


Figure 6: Buzzer Schematic

The buzzer uses a MOSFET circuit identical to that of the pyro circuitry, just without the resistance sensing segment.

HX711 Load Cell Amplifier



A [breakout board](#) was used for the HX711. The load cell that was in use when ARIES was made used only the E+, E-, and A- pins of the HX711 with 1k Ω resistors, so these resistors were implemented with jumpers that could be used to short them out in the future if necessary for use with a different load cell. The test stand includes an array of LEDs, which is controlled by another MOSFET, as well as the four HX711 pins. The schematic includes an optional current-limiting resistor for the LEDs, which can be shorted by a jumper if not needed.

PCB Layout

The PCB layout, as well as a 3D rendering (lacking several components' 3D models), can be seen below:

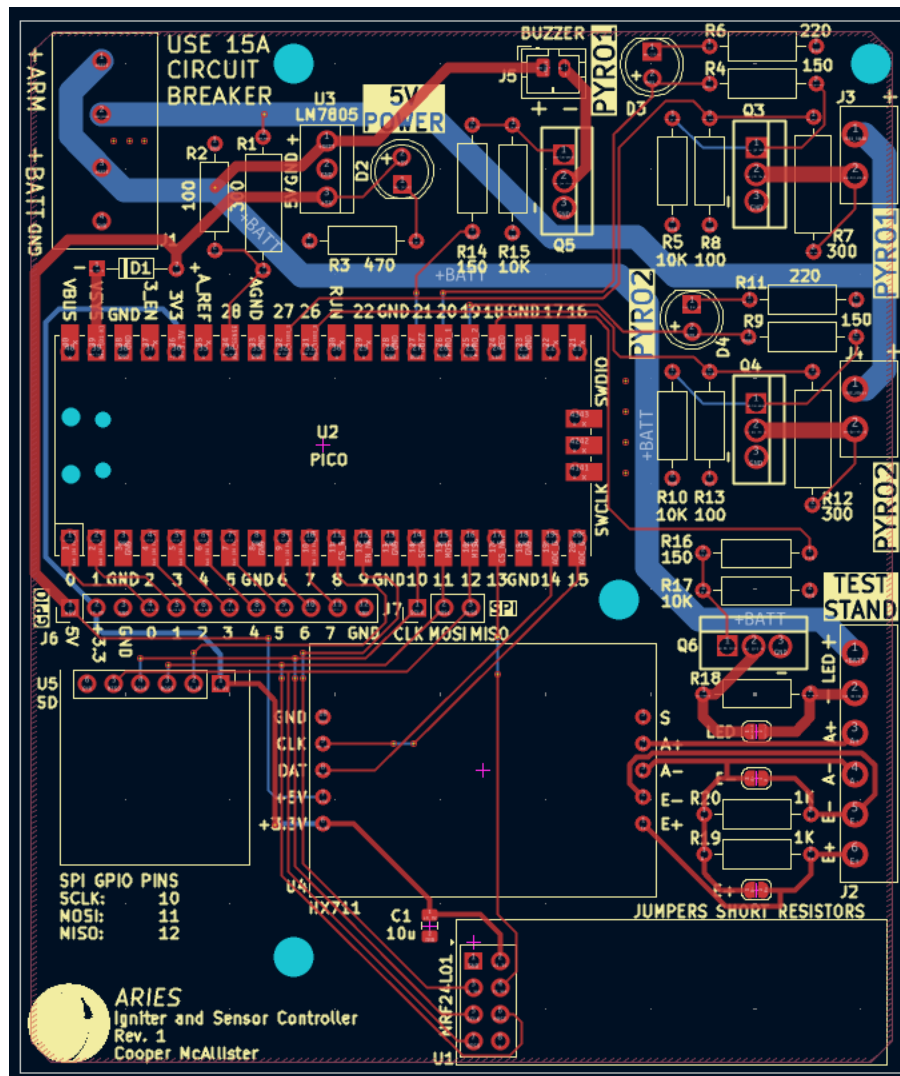


Figure 7: PCB Layout

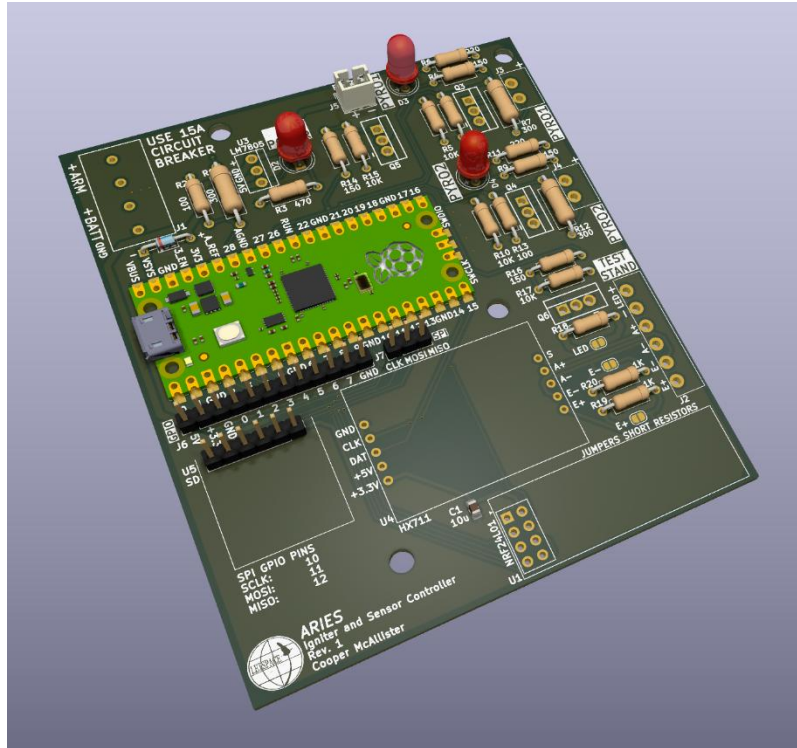


Figure 8: PCB Rendering

The PCB is 2 layers. The layout was designed so that the pyro and test stand connectors and radio antenna are on one side of the PCB, and the battery and arm switch connectors and Pi USB port are on the opposite side. A ground fill was used on both layers. Since most of the signals are DC or relatively low speed, factors such as power filtering, component proximity, trace impedance, and EMI were not given much thought (this was also due to my limited PCB design experience at the time). These factors would need to be considered for most other PCB designs.



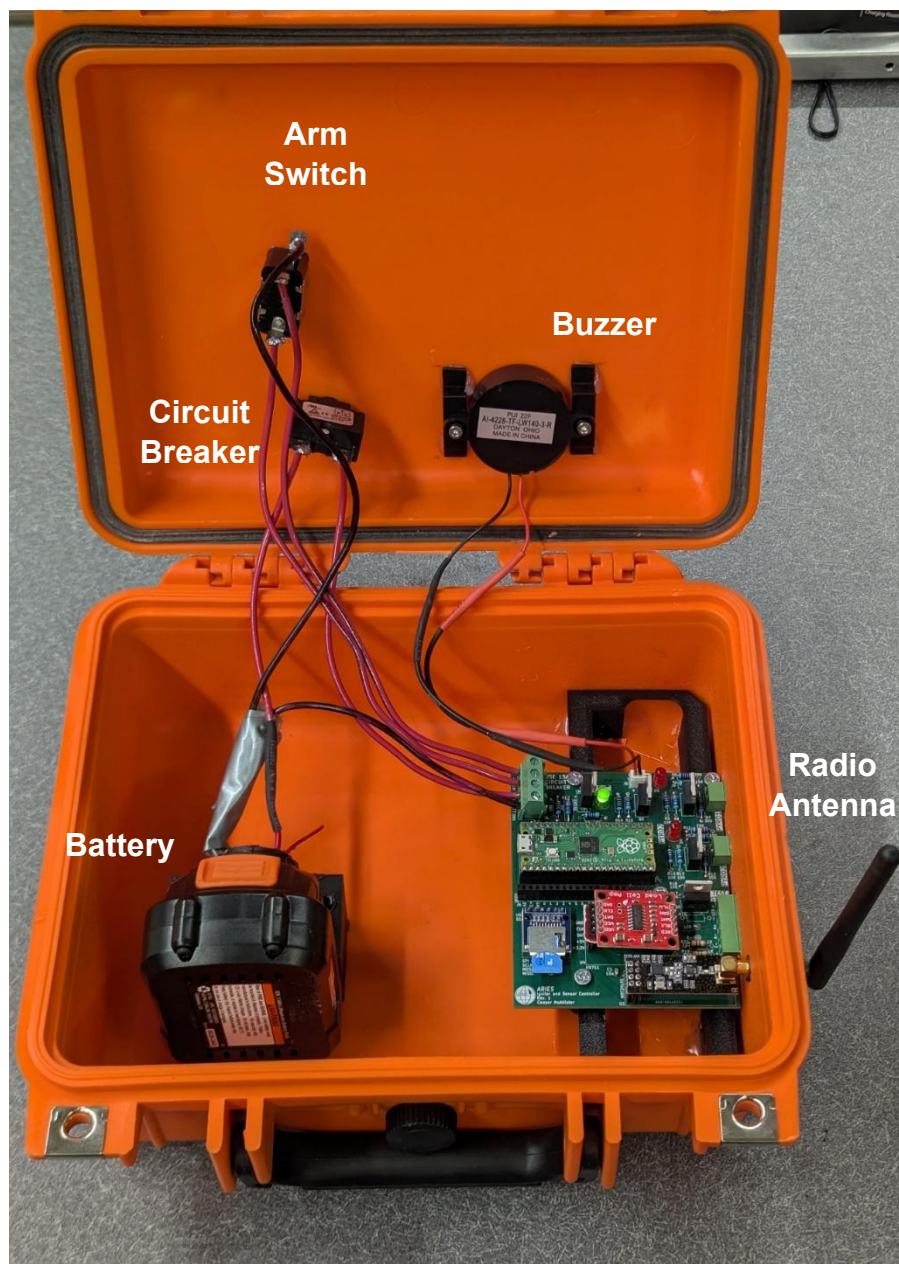
Hardware Design and Assembly

Besides the PCB, the case also contains the arm switch, circuit breaker, and battery. For the battery, I elected to use a battery from an [electric drill](#), and remove the bottom part of the drill to use as a battery mount.



Figure 9: Battery Mount

The entire case is shown below:





Software Design

ARIES was programmed in C using the [Raspberry Pi SDK](#) in VS Code, while CRIS was programmed using the Arduino IDE. Both pieces of software are in a GitHub repository for version control.

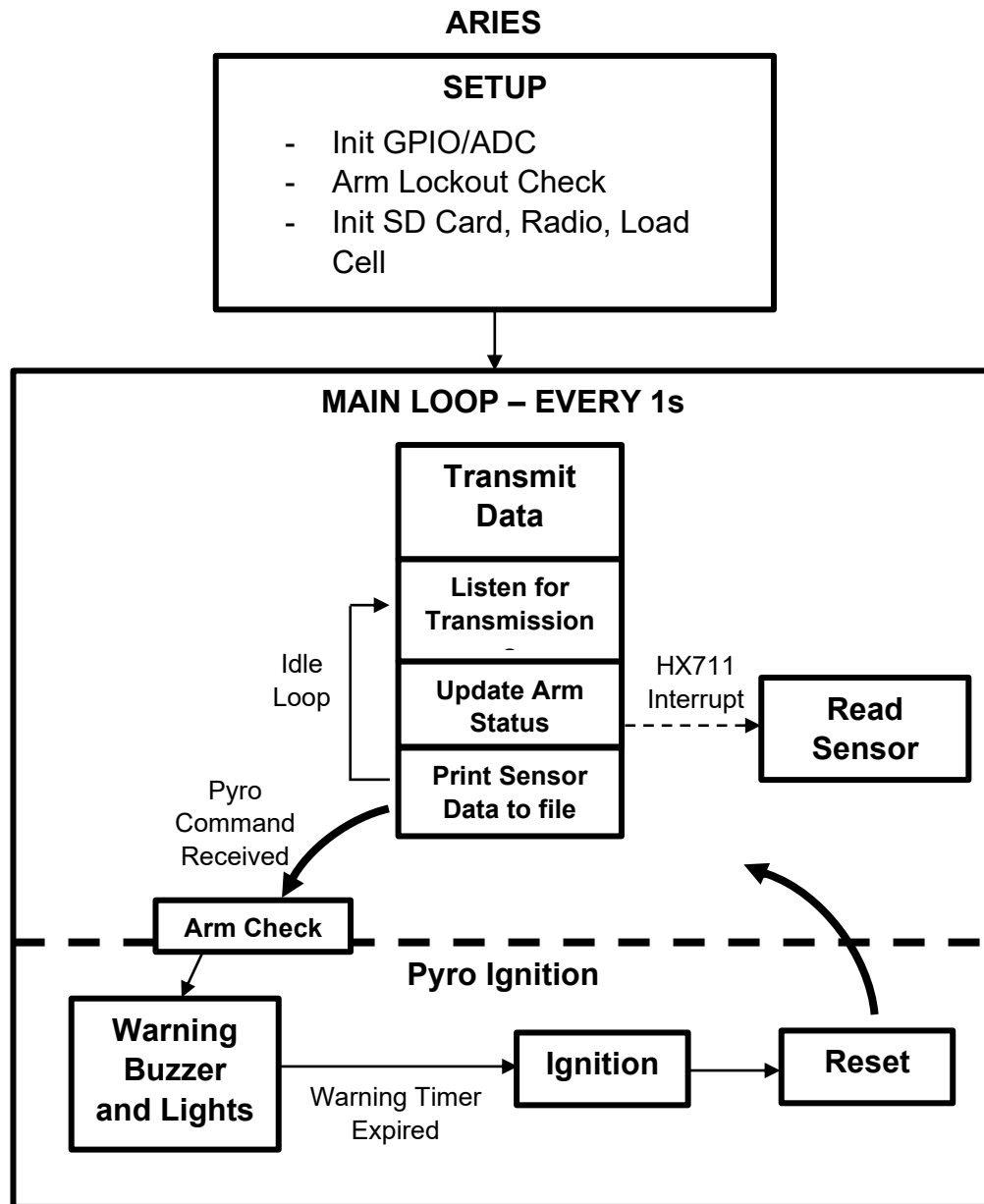
Following are instructions on how to set up the development environment:

- Use Visual Studio Code (not Microsoft Visual Studio), which can be downloaded from <https://code.visualstudio.com/>
- In VS Code, install the following extensions or verify they are already installed:
 - C/C++
 - Raspberry Pi Pico
 - Cmake tools
- In VS Code, open a local folder on your computer where your files will live.
- Open the terminal using `ctrl+ ~` and clone the git repository using:
`git clone https://github.com/LetSpace/ARIES`
- Restart VS Code. If prompted, select ARIES/CMakeLists.txt for the CMake file.
- Hopefully at this point there will be a “run” button on the bottom toolbar of VS Code, near the right (not the one on the left). If not, good luck! You have now entered the world of VS Code configuration, which I don’t really understand. The lack of the run button likely means that the Pico extension isn’t recognizing the project as a pico project. The extension is pretty particular about having the proper configuration. Double check that the proper extensions are installed, try rebuilding the project using `cmake`, etc.



Software Overview

The following block diagram outlines the software of ARIES and CRIS. In the code, CRIS is sometimes referred to as ARC (ARIES Remote Controller).





CRIS (ARC)

SETUP

- Init GPIO, radio, LCD
- Arm Lockout Check

MAIN LOOP – CONTINUOUS

Check for new
Button Data

If armed and ignition
button pressed

Wait for release

Confirm ignition
with 2nd button
press

Send ignition
command

Receive data and
update LCD

If no transmission
received for at least 2s

Display “no
connection”



Using ARIES

Below are some general instructions and guidelines on using ARIES. This should not be substituted for a proper static fire test procedure.

- ARIES and CRIS will automatically connect when both powered on. If not connected, the message “No Connection!” will appear on CRIS. When the system is used at the rugby field, the connection is somewhat spotty at the full distance (potentially due in part to the bleachers in the line of sight), so “No Connection!” flashing on and off is expected.
- Both ARIES and CRIS have an arm switch lockout; this means that the arm switch must be in the “disarmed” position to turn the device on. Arm lockout is indicated by the LCD on CRIS and by continuous buzzing by ARIES.
- When CRIS is disarmed, the ignition buttons can be pressed and won’t do anything. However, care should still be taken in handling CRIS to promote a practice of safety.
- While ARIES is armed, it will beep every few seconds, and the light on the arm switch will be lit.
- To ignite a pyro channel, press the button on CRIS corresponding to the channel you want to ignite. The button must be released and then pressed again to confirm ignition. If the button is not pressed for the second time within 10s, the ignition attempt will time out. After the button is pressed twice, ARIES will begin a 10s warning countdown. During the countdown, the lights on the test stand will flash, the buzzer will buzz, and CRIS will display the countdown time where it normally displays the status of ARIES.
- The resistance of each channel is shown on CRIS in $m\Omega$. When ARIES is disarmed, this value is meaningless. When ARIES is armed, an open channel will display “*****”, and a channel with an igniter connected will display the approximate resistance of the igniter. A good igniter is usually about 1000 $m\Omega$.
- The value of the load cell is displayed on CRIS in grams-force. This value is tared to 0 when ARIES starts, so to get a reading with no offset, ARIES should be turned on after the motor is set up on the load cell. The accuracy of the load cell is known to be about $\pm 100g$ at loads under about 4kg. The accuracy is not known at higher loads at the time of writing.
- There is an unpredictable software bug that sometimes causes ARIES to stop receiving transmissions after it has received three transmissions. This should not affect most tests since only one transmission is necessary for ignition, but it is good to know in case ARIES stops receiving transmissions.



Future Work and Lessons Learned

ARIES was overall a success, providing a reliable way to ignite and evaluate motors. However, there were several things that could have gone better with regard to planning and design:

- Deprioritization of hardware/mechanical design: I mostly focused on the PCB, and things such as designing the case or even adding mounting holes to the PCB were left for last. Better up-front hardware planning would have made a significant part of the project go smoother.
- Software development: Software features were added incrementally, on top of each other, with too little regard for big-picture planning to start with. Additionally, the development environment in VS code is difficult and unreliable to set up.

Additionally, there are some technical errors/potential improvements:

- Radio reliability: the NRF24L01 radios are finicky with regard to power supply and have choppy, but usable, performance at the range we need.
- The footprint for the Sparkfun HX711 breakout board on the PCB is 0.1in (2.54mm, aka one dupont header width) too wide. This is because the breakout was ordered at the same time as the PCB and the specifications on the website did not show the exact pitch of the headers, so an educated guess had to be made.
- Software multitasking: handling of multiple simultaneous tasks (receiving data, timing transmissions, recording data, timing warning and ignition) was thrown together by hand. It works, but a Real Time Operating System (RTOS) would have probably made development easier and the code simpler.
- Schottky diode: the Schottky diodes ordered for the project were the wrong size, so I used one I already had on hand, and it evidently has a high enough reverse leakage current to light the 5V power indicator when only USB power is connected to the board, which is slightly misleading.

On the whole, ARIES taught me a lot about seeing a project through from beginning to end, following time and budget restrictions, and technical concepts, including PCB design, embedded programming, and CAD design. With the help of ARIES, LetSpace will now be able to test motors and collect useful data to assist with propulsion development.